

自然语言处理基础（FNL P）课程笔记 v4

AutoPku / Codex

2026-04-28

Contents

说明	3
阅读路线	4
课程知识图谱	5
文件	6
v4 来源索引	7
第一章 课程图谱：NLP 任务、概率建模与结构化预测	8
1.1 自然语言处理的基本对象	8
1.2 语言为什么难	8
1.3 概率建模的统一句式	8
1.4 模型学习、解码与评价	9
第二章 词义消歧：从语义歧义到分类问题	10
2.1 WSD 的任务定义	10
2.2 Sense inventory 与 WordNet	10
2.3 三种学习范式	10
2.4 Lesk 与基线	10
2.5 WSD 评价	11
第三章 朴素贝叶斯与生成式分类	12
3.1 从监督学习到概率分类	12
3.2 生成式分类推导	12
3.3 Naive Bayes for WSD	12
3.4 独立性假设的意义	13
第四章 特征表示与 Log-linear 判别式模型	14
4.1 特征：证据的函数化	14
4.2 Bag-of-words 与 TF-IDF	14
4.3 线性打分模型	14
4.4 条件对数似然与梯度	15
4.5 正则化	15
第五章 n-gram 语言模型	16
5.1 语言模型要解决什么	16
5.2 链式法则与 Markov 假设	16
5.3 参数估计	17
5.4 Perplexity	17
5.5 未见事件、插值与平滑	17
第六章 神经语言模型与词表示入口	18
6.1 语言建模作为分类	18
6.2 前馈神经语言模型	18

6.3 CNN、RNN、LSTM、GRU	18
6.4 神经 LM 的训练与评估	18
第七章模型评估、数据切分与预训练范式	20
7.1 Train/Dev/Test	20
7.2 交叉验证	20
7.3 预训练与微调	20
第八章分布式语义与词向量	21
8.1 从 WordNet 到分布式语义	21
8.2 共现矩阵与 PPMI	21
8.3 SVD 与低维表示	21
8.4 Word2Vec 与负采样	21
8.5 GloVe 与全局统计	22
第九章 序列标注与 HMM	23
9.1 序列标注任务	23
9.2 POS、tagset 与 BIO	23
9.3 HMM 的生成故事	23
9.4 参数估计	23
9.5 Viterbi 解码	24
9.6 Forward 与 Baum-Welch	24
第十章 神经序列标注、CRF 与 Attention	25
10.1 从 HMM 到 MEMM/CRF	25
10.2 BiLSTM-CRF	25
10.3 Attention 的序列建模意义	25
第十一章 成分句法、PCFG 与 CKY	27
11.1 成分结构	27
11.2 CFG 与 PCFG	27
11.3 CNF 与 CKY	27
11.4 评价	28
第十二章依存句法	29
12.1 成分与依存的区别	29
12.2 依存关系与图约束	29
12.3 Transition-based parsing	29
12.4 Graph-based parsing	29
12.5 评价	30
第十三章 Seq2Seq、Attention 与 Transformer	31
13.1 Sequence-to-Sequence	31
13.2 Attention for Encoder-Decoder	31
13.3 Transformer	31
13.4 解码策略	32
13.5 BLEU 与生成评价	32
第十四章全课程公式与算法索引	33
14.1 公式总表	33
14.2 动态规划模板	33
14.3 易混点	33
14.4 最后一轮自测	33

说明

这是重新整理的 v4，不沿用 notes_v2 或 notes_v3 的分讲正文。整理依据是 notes/raw_text/ 中从课件抽取出的文本，覆盖 lec0 至 lec8、lec6_seq 以及李宏毅--seq2seq_v9.pptx。

本文按课程顺序重写为一份完整讲义：每讲保留课件中的任务动机、模型形式、推导主线、算法模板、评价指标和易错点，删去重复动画页、纯目录页和无助于复习的背景噪声。

阅读路线

目标	建议读法
快速建立全局框架	先读第 1 章课程图谱，再看每讲末尾速查
准备推导题	重点读 Naive Bayes、log-linear、n-gram、HMM、CKY、attention
准备算法题	重点读 Viterbi、Forward、CKY、依存 transition、beam search
准备概念题	对照生成式/判别式、MEMM/CRF、成分/依存、AT/NAT

课程知识图谱

```
graph TD
    A[导论：NLP 任务与歧义] --> B[WSD：分类任务入口]
    B --> C[Naive Bayes：生成式分类]
    B --> D[Log-linear：判别式分类]
    C --> E[n-gram 语言模型]
    D --> F[评估与正则]
    E --> G[神经语言模型]
    E --> H[分布式语义]
    H --> I[HMM 序列标注]
    I --> J[神经序列标注与 CRF]
    A --> K[PCFG 与 CKY]
    K --> L[依存句法]
    J --> M[Seq2Seq 与 Attention]
    G --> M
    M --> N[Transformer]
```

文件

- 主笔记: `fnlp_v4_full.md`
- 来源索引: `sources.md`
- PDF: `../pdf/FNLP 课程笔记_v4.pdf`

v4 来源索引

本目录的 v4 根据以下原始课件抽取文本重新整理：

v4 章节	原始文件
课程概览与 NLP 导论	notes/raw_text/lec0.pdf.txt, notes/raw_text/lec1_introduction.pdf.txt
词义消歧	notes/raw_text/lec2_wsd.pdf.txt
朴素贝叶斯与生成式分类	notes/raw_text/lec3_classification.pdf.txt
特征表示与 log-linear	notes/raw_text/lec3_loglinear.pdf.txt
n-gram 语言模型	notes/raw_text/lec4_lm.pdf.txt
神经语言模型	notes/raw_text/lec4_nlm.pdf.txt
模型评估与训练协议	notes/raw_text/lec4_pre.pdf.txt
分布式语义	notes/raw_text/lec5_dist.pdf.txt
序列标注与 HMM	notes/raw_text/lec6_hmm.pdf.txt
神经序列标注、attention、Transformer	notes/raw_text/lec6_seq.pdf.txt, notes/raw_text/李宏毅--seq2seq_v9.pptx.txt
成分句法与 PCFG	notes/raw_text/lec7_phrase.pdf.txt
依存句法	notes/raw_text/lec8_dp.pdf.txt

处理原则：

1. 课件中重复出现的动画页只保留一次。
2. 目录页、引用页、纯图片页只在影响结构时保留。
3. 公式按课程中的概率建模顺序重新排版。
4. 例子保留具有教学意义的，如 bank WSD、speech recognition 的 recognize speech/wreck a nice beach、HMM/Viterbi、PCFG/CKY、Transformer attention。

第一章 课程图谱：NLP 任务、概率建模与结构化预测

[TIP] 学习指南

本章对应 lec0 与 lec1_introduction。核心是建立整门课的任务地图：NLP 不是一组孤立模型，而是在不同输出空间上做预测、打分和搜索。

1.1 自然语言处理的基本对象

自然语言处理 (Natural Language Processing, NLP) 研究怎样让计算系统处理人类语言。课件从课程安排进入主题，但真正的主线很快出现：语言既有词法、句法、语义、语用等层次，也有工程上明确的输入输出。一个 NLP 系统通常不是直接“理解语言”，而是在某个任务定义下把输入 x 映射到输出 y ，并用数据学习这个映射。

课程中的任务可以按输出空间分为四类。第一类是单点分类，例如词义消歧、文本分类；第二类是序列概率建模，例如语言模型；第三类是等长序列标注，例如词性标注、命名实体识别、中文分词；第四类是结构化输出，例如成分句法树、依存图、机器翻译输出序列。输出空间越复杂，模型越需要显式处理全局一致性和搜索。

1.2 语言为什么难

课件导论反复强调歧义。歧义可能出现在词义、词性、短语附着、句法结构、指代、语用意图等层面。一个词可以有多个义项，一句话可以有多种句法树，一段文本的真实意图还可能依赖上下文和世界知识。课程后面引入的分类模型、语言模型、HMM、PCFG、依存句法和 Transformer，本质上都是不同抽象层级上压缩、利用和消解歧义。

另一个困难是组合爆炸。有限词表可以组成无限多句子，有限标签集可以形成指数多标签序列，有限文法规则可以生成大量树结构。暴力枚举通常不可行，因此课程大量使用独立性假设和动态规划：n-gram 用 Markov 假设压缩历史，HMM 用局部转移压缩标签序列，CKY 用 span chart 复用子结构，Transformer 用注意力在可并行结构中建模长距离依赖。

1.3 概率建模的统一句式

整门课可以用三种概率对象串起来。

第一种是条件分布：

$$p(y \mid x).$$

分类、序列标注、句法分析和机器翻译最终都关心给定输入 x 后最可能的输出 y 。判别式模型直接学习这个对象。

第二种是联合分布：

$$p(x, y) = p(y)p(x \mid y).$$

朴素贝叶斯、HMM、PCFG 都先讲一个生成故事，再用 Bayes rule 或最大联合概率做预测。

第三种是序列概率：

$$p(w_{1:n}) = \prod_{i=1}^n p(w_i \mid w_{1:i-1}).$$

语言模型围绕这个对象展开，后续神经语言模型、Seq2Seq decoder 和自回归生成都继承这一形式。

1.4 模型学习、解码与评价

课程中每个模型都要回答三件事。第一，参数如何估计：计数式 MLE、条件对数似然、正则化最大似然、负采样、EM 或神经网络反向传播。第二，预测时如何解码：分类取最大分数，HMM 用 Viterbi，语言模型逐步预测，PCFG 用 CKY，依存句法用 transition 或 graph search，Seq2Seq 用 greedy、beam 或 sampling。第三，怎样评价：分类看 accuracy/F1，语言模型看 perplexity，句法看 PARSEVAL/UAS/LAS，生成看 BLEU 或任务指标。

[WARN] 易错点

模型名字不是复习的中心。先问输出是什么，再问概率对象是什么，最后问训练目标和解码算法。这样遇到新模型也能放回课程框架里。

第二章词义消歧：从语义歧义到分类问题

[TIP] 学习指南

本章对应 lec2_wsd。它是课程进入机器学习模型的入口：给定目标词及上下文，从固定义项清单中选出正确义项。

2.1 WSD 的任务定义

词义消歧 (Word Sense Disambiguation, WSD) 的输入是目标词 w 、上下文 C 和义项集合 $S = \{s_1, \dots, s_K\}$ ，输出是目标词在该上下文中的义项：

$$s^* = \arg \max_{s \in S} \text{Score}(s, C).$$

课件从“words may have multiple distinct meanings”出发，区分同形异义和一词多义。英语 **bank** 可以指金融机构，也可以指河岸；汉语中“长”在“长高”和“很长”里读音和意义都不同。WSD 不是只为词典服务，它直接影响语音合成、机器翻译、信息抽取、问答和实体消歧。

2.2 Sense inventory 与 WordNet

WSD 必须先规定候选义项。WordNet 是课件重点介绍的英文义项资源：它以 synset 组织名词、动词、形容词和副词的词义，并给出 gloss、例句、同义词、上位词、下位词等关系。对 WSD 来说，WordNet 的作用是定义标签空间；对 Lesk 这类知识库方法来说，它还提供了可比较的释义文本。

义项清单也带来困难。清单越细，模型越难预测，标注一致性越差；清单越粗，又可能不能满足真实任务需求。因此 WSD 既是机器学习问题，也是任务定义问题。

2.3 三种学习范式

监督式 WSD 把问题看成多分类：训练集包含 $(x^{(i)}, y^{(i)})$ ，其中 $x^{(i)}$ 是含目标词的上下文， $y^{(i)}$ 是义项标签。模型从上下文抽取特征，学习 $g(x) = y$ 或 $p(y | x)$ 。

半监督式 WSD 使用少量种子或高置信预测扩充标注。课件中的 Yarowsky 思路依赖两个经验规律：one sense per discourse 与 one sense per collocation。同一文档中的同一个词倾向共享义项，固定搭配也倾向强烈指向一个义项。

无监督式 WSD 更准确地说是词义归纳 (Word Sense Induction, WSI)：模型从上下文相似性中聚类出不同用法，但聚类结果还需要映射回人工义项清单。

2.4 Lesk 与基线

Most Frequent Sense (MFS) 是 WSD 的强基线：永远选训练语料中最常见的义项。它简单但很难被弱模型超过，因为自然语言中的义项分布通常长尾且偏斜。

Lesk 算法不依赖标注语料。它比较上下文词与候选义项 gloss 的重叠：

$$s^* = \arg \max_{s \in S} |G(s) \cap C|.$$

扩展版 Lesk 会加入上位词、下位词 gloss，或给词重叠加权。Lesk 的核心假设是：正确义项的释义词更可能与上下文词共享语义线索。

2.5 WSD 评价

WSD 常用 accuracy、precision、recall、F1。若是 all-words setting，模型需要对文本中所有开放类词逐一消歧；若是 lexical-sample setting，只对预先指定的目标词消歧。多类 F1 可以 macro-average，也可以 micro-average。macro 更关注小类，micro 更受大类影响。

[TIP] 本讲速查

WSD = 固定义项清单 + 上下文特征 + 分类/匹配。WordNet 给标签空间，MFS 给强基线，Lesk 用 gloss 重叠，监督方法用标注数据学习 $p(s \mid C)$ 或打分函数。

第三章朴素贝叶斯与生成式分类

[TIP] 学习指南

本章对应 lec3_classification。它承接 WSD，说明如何从 $p(y | x)$ 目标转向生成式建模 $p(x, y)$ 。

3.1 从监督学习到概率分类

监督学习给定训练样本：

$$(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)}),$$

目标是学习函数 $g : x \mapsto y$ 。概率视角下，分类器选择：

$$g(x) = \arg \max_y p(y | x).$$

课件提出两个视角：判别式模型直接学习 $p(y | x)$ ；生成式模型先学习 $p(x, y)$ ，再由 Bayes rule 推出分类规则。

3.2 生成式分类推导

Bayes rule 给出：

$$p(y | x) = \frac{p(x, y)}{p(x)} = \frac{p(x | y)p(y)}{\sum_{y'} p(x | y')p(y')}.$$

对固定 x ，分母与候选标签无关，因此：

$$\arg \max_y p(y | x) = \arg \max_y p(x | y)p(y) = \arg \max_y p(x, y).$$

这就是生成式分类器的通用形式。模型讲一个“先产生标签 y ，再根据标签产生观察 x ”的故事，然后估计先验 $p(y)$ 和似然 $p(x | y)$ 。

3.3 Naive Bayes for WSD

在 WSD 中， y 是义项 s ， x 是上下文 C 。朴素贝叶斯的目标是：

$$s^* = \arg \max_{s_k} P(C | s_k)P(s_k).$$

如果上下文由特征词 v_x 构成，Naive Bayes 作条件独立假设：

$$P(C | s_k) = \prod_{v_x \in C} P(v_x | s_k).$$

最终：

$$s^* = \arg \max_{s_k} P(s_k) \prod_{v_x \in C} P(v_x | s_k)$$

参数由计数估计：

$$P(v_x | s_k) = \frac{\text{Count}(v_x, s_k)}{\sum_{v \in V} \text{Count}(v, s_k)}, \quad P(s_k) = \frac{\text{Count}(s_k)}{\text{Count}(w)}.$$

3.4 独立性假设的意义

条件独立假设在语言中明显不成立。词与词之间会共现，语法位置和语义场也会相关。但该假设让参数数量从难以估计的联合上下文分布降为每个特征在每个类别下的条件概率。课程中“THIS IS NOT TRUE! but we then have an easier model”这句话就是本章重点：好的概率模型不总是完全真实，而是要在可估计性和表达能力之间折中。

实际实现时通常使用 log space：

$$s^* = \arg \max_{s_k} \left[\log P(s_k) + \sum_{v_x \in C} \log P(v_x | s_k) \right],$$

避免大量小概率相乘造成数值下溢。

[TIP] 本讲速查

Naive Bayes 是生成式模型。推导顺序是 $p(y | x)$ 到 $p(x | y)p(y)$ ，再用条件独立把 $p(x | y)$ 拆成特征似然乘积。

第四章 特征表示与 Log-linear 判别式模型

[TIP] 学习指南

本章对应 lec3_loglinear。课件从 WSD 的“线索”出发，转向特征函数、线性打分、条件概率归一化和正则化。

4.1 特征：证据的函数化

课件把 **feature** 定义为描述观察数据 x 与候选标签 y 之间某些方面的证据。一个特征通常写为：

$$f_i(x, y) \in \mathbb{R}.$$

在 NLP 中它常是二值 indicator。例如对 bank 做 WSD：

$$f_i(x, y) = \begin{cases} 1, & \text{if previous word is transfer and } y = \text{FINANCIAL}, \\ 0, & \text{otherwise.} \end{cases}$$

特征可以来自目标词、前后词、POS、前后缀、大小写、句法关系，也可以是 bag-of-words、TF-IDF、dense embedding。课程中新闻分类例子说明，同一篇文本在不同候选标签下会激活不同的 label-specific word features。

4.2 Bag-of-words 与 TF-IDF

Bag-of-words 把文档表示为词表维度上的向量，忽略顺序。它可以是 0/1 one-hot 出现向量，也可以是词频向量，还可以用 TF-IDF 加权：

$$\text{TF}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t, d), & \text{count}(t, d) > 0, \\ 0, & \text{otherwise,} \end{cases}$$

$$\text{IDF}_t = \log_{10} \frac{N}{\text{DF}_t}, \quad \text{TFIDF}_{t,d} = \text{TF}_{t,d} \text{IDF}_t.$$

TF-IDF 的直觉是：在当前文档中频繁出现的词重要，但在所有文档中都常见的词要降权。它是许多线性文本分类模型的基础输入。

4.3 线性打分模型

最简单的特征判别式模型对每个候选标签打分：

$$\text{score}(x, y) = \sum_i \lambda_i f_i(x, y) = \lambda^\top f(x, y).$$

预测：

$$y^* = \arg \max_y \text{score}(x, y).$$

这已经能做分类，但它还不是概率模型。log-linear 模型把线性分数指数化并归一化：

$$p_{\theta}(y | x) = \frac{\exp(\theta^{\top} f(x, y))}{Z_{\theta}(x)}$$

其中：

$$Z_{\theta}(x) = \sum_{y'} \exp(\theta^{\top} f(x, y')).$$

4.4 条件对数似然与梯度

单个样本的条件对数似然：

$$\ell(\theta) = \log p_{\theta}(y | x) = \theta^{\top} f(x, y) - \log Z_{\theta}(x).$$

对参数 θ_k 求导：

$$\frac{\partial \ell}{\partial \theta_k} = f_k(x, y) - \sum_{y'} p_{\theta}(y' | x) f_k(x, y').$$

全数据上就是：

$$\nabla_{\theta} L = \text{empirical feature count} - \text{expected feature count}$$

这说明训练的目标是让模型期望特征计数靠近训练数据中的经验特征计数。最大熵模型的 moment matching 直觉就来自这里。

4.5 正则化

如果某个特征和某个标签在训练集中完全共现，未正则化模型可能把对应权重推向无穷大。L2 正则则在目标函数中加入：

$$L_{\text{reg}}(\theta) = L(\theta) - \frac{\alpha}{2} \|\theta\|^2,$$

梯度变为：

$$\nabla L_{\text{reg}} = \nabla L - \alpha \theta.$$

正则化的作用是限制参数规模，降低过拟合，并让权重在证据不足时回到较小值。

[TIP] 本讲速查

Log-linear = 特征函数 + 线性分数 + 指数化 + 条件归一化。最重要推导是 $\nabla = \text{empirical} - \text{expected}$ ，最重要工程问题是正则化。

第五章 n-gram 语言模型

[TIP] 学习指南

本章对应 lec4_lm。语言模型给词序列分配概率，是后续神经 LM 和生成模型的基础。

5.1 语言模型要解决什么

课件用 “the dog barks”、“the dog laughs”、“dog the bark a laugh smile” 引出语言模型：给定一个词串，判断它像不像某种语言中的合法表达。形式化地，语言模型是在所有词序列 S 上定义概率分布：

$$\sum_{s \in S} p(s) = 1, \quad p(s) \geq 0.$$

好句子应获得较大概率，坏句子应获得较小概率。

语言模型在语音识别、OCR、手写识别、中文分词、机器翻译、语言生成、问答和对话中都重要。课件中的 recognize speech / wreck a nice beach 例子说明，声学或视觉输入可能给出多个候选，语言模型负责选择更像自然语言的那个。

5.2 链式法则与 Markov 假设

对句子 $w_{1:n}$ ：

$$p(w_{1:n}) = \prod_{i=1}^n p(w_i \mid w_{1:i-1}).$$

完整历史太长，无法可靠估计。n-gram 作 Markov 近似：

$$p(w_i \mid w_{1:i-1}) \approx p(w_i \mid w_{i-n+1:i-1}).$$

常见模型：

模型	假设	形式
unigram	不看历史	$p(w_i)$
bigram	看前 1 个词	$p(w_i \mid w_{i-1})$
trigram	看前 2 个词	$p(w_i \mid w_{i-2}, w_{i-1})$

trigram 句子概率：

$$p(w_{1:n}) = p(w_1)p(w_2 \mid w_1) \prod_{i=3}^n p(w_i \mid w_{i-2}, w_{i-1}).$$

5.3 参数估计

MLE 用计数比：

$$\hat{p}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})},$$
$$\hat{p}(w_i | w_{i-2}, w_{i-1}) = \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})}.$$

课件用 Gigaword 规模说明参数空间极大且稀疏。即使 10 million sentences、275 million words，很多可能 n-gram 也从未出现。MLE 对未见事件给零概率，这会让包含未见 n-gram 的测试句概率变成零。

5.4 Perplexity

语言模型在测试集 D 上的目标是给真实句子高概率。交叉熵：

$$H(D) = -\frac{1}{M} \sum_{s \in D} \log_2 p(s),$$

其中 M 是测试集 token 数。困惑度：

$$\text{PPL}(D) = 2^{H(D)} = 2^{-\frac{1}{M} \sum_{s \in D} \log_2 p(s)}$$

PPL 越低，模型越好。直觉上，perplexity 是模型平均每步面对的有效候选数。

5.5 未见事件、插值与平滑

为避免零概率，需要把概率质量分给未见事件。线性插值把不同阶模型混合：

$$p(w_i | w_{i-2}, w_{i-1}) = \lambda_3 p_{\text{tri}} + \lambda_2 p_{\text{bi}} + \lambda_1 p_{\text{uni}},$$

其中 $\lambda_1 + \lambda_2 + \lambda_3 = 1$ 。

常见平滑包括 add-one、Good-Turing、absolute discounting、Kneser-Ney。Add-one 简单但常过度平滑；Good-Turing 用低频事件估计未见质量；Kneser-Ney 的关键是 continuation probability：一个词作为新上下文延续的能力，比它的总频数更能解释某些 bigram 概率。

[TIP] 本讲速查

n-gram LM = 链式法则 + Markov 假设 + 计数估计 + 平滑。考试最常见是算 trigram 概率、PPL，解释数据稀疏和平滑动机。

第六章神经语言模型与词表示入口

[TIP] 学习指南

本章对应 lec4_nlm。课件把语言建模重新看成分类问题：给定上下文，预测下一个词；神经网络的关键收益是用 dense representation 缓解稀疏。

6.1 语言建模作为分类

神经语言模型把每个位置的预测写为：

$$p(w_t | w_{<t}) = \text{softmax}(g(h_t)),$$

其中 h_t 是上下文表示，输出维度是词表大小。相比 n-gram 直接查表，神经 LM 学习共享参数：相似上下文和相似词可以通过向量空间共享统计强度。

6.2 前馈神经语言模型

早期 NNLM 使用固定窗口。例如给定 $w_{t-k}, \dots, w_{t-1}$ ，先查词向量，再拼接或平均，经过非线性层预测 w_t ：

$$h = \sigma(W[e(w_{t-k}); \dots; e(w_{t-1})] + b),$$

$$p(w_t | h) = \text{softmax}(Uh + c).$$

它比 n-gram 好的地方是：参数不再为每个离散 history 独立存储，而是通过 embedding 和网络权重共享。

6.3 CNN、RNN、LSTM、GRU

CNN 可以用卷积窗口抽取局部 n-gram-like 模式。RNN 用递归状态表示任意长度历史：

$$h_t = \phi(W_x e(w_t) + W_h h_{t-1} + b).$$

普通 RNN 面临梯度消失/爆炸。LSTM 和 GRU 通过门控机制控制记忆写入、遗忘和输出，使模型更适合长距离依赖。

6.4 神经 LM 的训练与评估

训练目标通常是最大化语料 log-likelihood：

$$\sum_t \log p_\theta(w_t | w_{<t}).$$

评估仍使用 perplexity。注意：PPL 是语言模型内在指标，不一定完全等价于下游任务性能；但在同一数据和词表设置下，低 PPL 通常表示更好的预测能力。

[TIP] 本讲速查

神经 LM 的核心变化是从稀疏 history table 转为 dense context representation。它仍在做下一个词分类，仍用 softmax 概率和 PPL 评价。

第七章模型评估、数据切分与预训练范式

[TIP] 学习指南

本章对应 lec4_pre。课件文本抽取中噪声较多，但核心主题是训练、验证、测试以及预训练/微调的实验协议。

7.1 Train/Dev/Test

机器学习实验必须区分训练集、验证集和测试集。训练集用于拟合参数；验证集用于选择超参数、特征、模型结构和早停点；测试集只用于最终报告。若频繁根据测试集调整模型，就会把测试集泄漏为开发集，得到过于乐观的估计。

7.2 交叉验证

当数据较少，可以使用 K 折交叉验证：把数据分成 K 份，每次用一份作验证，其余作训练，重复 K 次后平均结果。它能降低单次划分带来的方差，但训练成本更高。

7.3 预训练与微调

预训练 (pre-training) 先在大规模通用语料上学习表示或语言模型，再在下游任务上微调 (fine-tuning)。从课程主线看，它延续了神经语言模型：语言模型训练提供可迁移的词或上下文表示，下游任务只需少量标注即可利用这些表示。

[WARN] 易错点

不能用测试集调参。**dev set** 的作用是替代测试集参与模型选择；**test set** 的作用是估计最终泛化性能。

第八章分布式语义与词向量

[TIP] 学习指南

本章对应 lec5_dist。它从 WordNet 等符号语义资源转向 distributional hypothesis: 相似上下文中的词倾向意义相似。

8.1 从 WordNet 到分布式语义

WordNet 用人工语义关系组织词义, 优点是结构清晰, 缺点是覆盖、更新和粒度都受人工维护限制。分布式语义从语料中学习词义表示, 核心假设是:

words that appear in similar contexts tend to have similar meanings.

因此, 词的意义可以由它出现的上下文分布来近似。

8.2 共现矩阵与 PPMI

构造词-上下文矩阵 X , 其中 X_{wc} 表示词 w 与上下文 c 的共现次数。PMI 衡量二者共现是否超过独立假设:

$$\text{PMI}(w, c) = \log \frac{p(w, c)}{p(w)p(c)}.$$

PPMI 截断负值:

$$\text{PPMI}(w, c) = \max(\text{PMI}(w, c), 0).$$

PPMI 比 raw count 更关注有信息量的搭配, 但矩阵仍然高维稀疏。

8.3 SVD 与低维表示

对共现矩阵做低秩分解:

$$X \approx U_k \Sigma_k V_k^\top.$$

词向量可取 $U_k \Sigma_k$ 或相关变体。SVD 的意义是把高维稀疏共现压缩成低维 dense representation, 保留主要统计结构。

8.4 Word2Vec 与负采样

Word2Vec 包含 CBOW 和 Skip-gram。CBOW 用上下文预测中心词; Skip-gram 用中心词预测上下文。Skip-gram 目标可写为最大化:

$$\sum_{(w, c) \in D} \log p(c | w).$$

完整 softmax 成本高, 负采样把问题转化为二分类: 真实 (w, c) 对应正例, 随机采样 (w, c') 对应负例:

$$\log \sigma(v_c^\top v_w) + \sum_{j=1}^k \mathbb{E}_{c_j \sim P_n} \log \sigma(-v_{c_j}^\top v_w).$$

8.5 GloVe 与全局统计

GloVe 拟合词向量内积与共现计数的对数之间的关系：

$$J = \sum_{i,j} f(X_{ij})(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2.$$

它位于矩阵分解与局部预测之间：既利用全局共现矩阵，又学习 **dense word vectors**。

[TIP] 本讲速查

分布式语义从“词典关系”转向“上下文相似”。PPMI/SVD 是计数矩阵路线，Word2Vec 是预测路线，GloVe 是全局共现拟合路线。

第九章 序列标注与HMM

[TIP] 学习指南

本章对应 lec6_hmm。核心是把 POS tagging、NER、分词等任务统一为输入序列到标签序列的预测，并用 HMM 建模联合概率。

9.1 序列标注任务

序列标注给定 $x_{1:n}$ ，输出等长标签序列 $y_{1:n}$ 。典型任务包括 POS tagging、NER、chunking、中文分词、case restoration、accent restoration。与单点分类不同，标签之间存在依赖。例如 POS tag 的合法转移、BIO 标签的边界约束，都不能用独立分类完全表达。

9.2 POS、tagset 与 BIO

POS 描述词的句法类别。闭类词如 determiners、pronouns、prepositions，成员较固定；开类词如 nouns、verbs、adjectives、adverbs，成员可增长。tagset 是对这些类别的标准化编码，例如 Penn Treebank tagset 和 Universal POS。

BIO 把 span 识别转成 token-level labeling: B 表示开始，I 表示内部，O 表示外部。带类型时可写成 B-ORG、I-ORG、B-LOC 等。

9.3 HMM 的生成故事

HMM 是生成式序列模型，包含隐状态 y_t 和观察 x_t 。bigram HMM 参数：

参数	含义
$\pi(y)$	初始状态分布
$a_{ij} = p(y_t = j \mid y_{t-1} = i)$	转移概率
$b_j(x) = p(x_t = x \mid y_t = j)$	发射概率

联合概率：

$$p(x_{1:n}, y_{1:n}) = \pi(y_1) b_{y_1}(x_1) \prod_{t=2}^n a_{y_{t-1}, y_t} b_{y_t}(x_t).$$

trigram HMM 用 $p(y_t \mid y_{t-2}, y_{t-1})$ ，因此 Viterbi 状态要保留两个最近标签。

9.4 参数估计

有标签训练集上，HMM 参数用计数式 MLE：

$$\hat{p}(y_t \mid y_{t-1}) = \frac{c(y_{t-1}, y_t)}{c(y_{t-1})},$$

$$\hat{p}(x_t | y_t) = \frac{c(y_t, x_t)}{c(y_t)}.$$

这与 n-gram LM 的估计形式完全同构。实际需要平滑，并对 OOV 词设置<UNK> 或形态特征回退。

9.5 Viterbi 解码

目标：

$$\hat{y}_{1:n} = \arg \max_{y_{1:n}} p(x_{1:n}, y_{1:n}).$$

bigram Viterbi 定义：

$$\delta_t(j) = \max_{y_{1:t-1}} p(y_{1:t-1}, y_t = j, x_{1:t}).$$

递推：

$$\delta_t(j) = \max_i \delta_{t-1}(i) a_{ij} b_j(x_t)$$

回指针：

$$\psi_t(j) = \arg \max_i \delta_{t-1}(i) a_{ij}.$$

复杂度 $O(n|S|^2)$ 。

trigram Viterbi：

$$\pi(k, u, v) = \max_w \pi(k-1, w, u) p(v | w, u) p(x_k | v),$$

复杂度 $O(n|S|^3)$ 。

9.6 Forward 与 Baum-Welch

Forward 算法计算观察序列概率：

$$\alpha_t(j) = p(x_{1:t}, y_t = j).$$

递推：

$$\alpha_t(j) = \left[\sum_i \alpha_{t-1}(i) a_{ij} \right] b_j(x_t).$$

它与 Viterbi 的差别是 sum 与 max。无标签训练时，Baum-Welch 使用 EM 估计隐状态后验和参数。

[TIP] 本讲速查

HMM 三问：likelihood 用 Forward，decoding 用 Viterbi，learning 有标签用 MLE、无标签用 Baum-Welch。Viterbi 是 max，Forward 是 sum。

第十章 神经序列标注、CRF 与 Attention

[TIP] 学习指南

本章综合 lec6_seq 中序列模型部分。它把 HMM 的离散生成式序列建模推进到神经编码、CRF 全局归一和 attention。

10.1 从 HMM 到 MEMM/CRF

HMM 的局限是观察模型过弱： x_t 只依赖 y_t ，难以使用整句上下文特征。MEMM 改成每一步条件模型：

$$p(y_t | y_{t-1}, x) = \frac{\exp(\theta^\top f(y_{t-1}, y_t, x, t))}{\sum_{y'} \exp(\theta^\top f(y_{t-1}, y', x, t))}.$$

它能使用丰富特征，但局部归一导致 label bias。

Linear-chain CRF 对整条路径全局归一：

$$p(y_{1:n} | x) = \frac{\exp\left(\sum_t \theta^\top f(y_{t-1}, y_t, x, t)\right)}{\sum_{y'_{1:n}} \exp\left(\sum_t \theta^\top f(y'_{t-1}, y'_t, x, t)\right)}.$$

CRF 解码仍可用 Viterbi，训练中的归一化项用 Forward 类动态规划计算。

10.2 BiLSTM-CRF

神经序列标注常用 BiLSTM 编码输入：

$$h_t = [\vec{h}_t; \overleftarrow{h}_t].$$

每个位置得到 emission score，再加上标签转移分数：

$$\text{Score}(x, y) = \sum_t \text{emit}(h_t, y_t) + \sum_t T_{y_{t-1}, y_t}.$$

CRF 层用全局 softmax：

$$p(y | x) = \frac{\exp(\text{Score}(x, y))}{\sum_{y'} \exp(\text{Score}(x, y'))}.$$

这样既利用神经网络的上下文表示，又保留标签转移一致性。

10.3 Attention 的序列建模意义

Attention 让模型在当前决策时选择性读取其他位置。对 query q 、keys k_i 、values v_i ：

$$\alpha_i = \text{softmax}(q^\top k_i), \quad c = \sum_i \alpha_i v_i.$$

它解决了固定长度状态难以承载全部历史的问题，也为后续 Transformer 铺路。

[TIP] 本讲速查

HMM 用生成式局部假设，MEMM 用局部条件概率，CRF 用全局条件概率。BiLSTM-CRF 是神经 emission + CRF transition。

第十一章 成分句法、PCFG 与 CKY

[TIP] 学习指南

本章对应 lec7_phrase。核心是从词序列输出短语结构树，并用 PCFG 给树打概率，用 CKY 做动态规划解析。

11.1 成分结构

成分句法 (constituency parsing) 把句子分析成层级短语结构，如 NP、VP、PP。短语结构树强调一个片段如何作为整体在更大结构中发挥作用。它与后面的依存句法不同：成分句法关注 span 和短语类别，依存句法关注词与词之间的 head-dependent 关系。

11.2 CFG 与 PCFG

上下文无关文法 (CFG) 包含非终结符、终结符、规则和起始符号。规则形如：

$$A \rightarrow \beta.$$

PCFG 给每条规则概率：

$$\sum_{\beta} P(A \rightarrow \beta) = 1.$$

一棵树的概率是其中所有规则概率的乘积：

$$P(T) = \prod_{r \in T} P(r).$$

参数 MLE：

$$\hat{P}(A \rightarrow \beta) = \frac{c(A \rightarrow \beta)}{\sum_{\beta'} c(A \rightarrow \beta')}.$$

11.3 CNF 与 CKY

CKY 通常要求文法为 Chomsky Normal Form：规则为 $A \rightarrow BC$ 或 $A \rightarrow w$ 。定义 chart：

$$\pi[i, j, A] = \text{span}(i, j) \text{ 被 } A \text{ 推导的最高概率}.$$

初始化：

$$\pi[i, i+1, A] = P(A \rightarrow w_i).$$

递推：

$$\pi[i, j, A] = \max_{A \rightarrow BC} \max_{i < k < j} P(A \rightarrow BC) \pi[i, k, B] \pi[k, j, C]$$

答案在 $\pi[0, n, S]$ ，回指针记录最佳规则和切分点。

11.4 评价

成分句法常用 PARSEVAL：比较预测树和 gold tree 的 labeled constituents，计算 precision、recall、F1。句法树的边界和标签都重要，因此只预测出大致结构还不够。

[TIP] 本讲速查

PCFG = CFG 规则概率化；树概率 = 规则概率乘积；CKY = span chart + split point + grammar rule。

第十二章依存句法

[TIP] 学习指南

本章对应 lec8_dp。依存句法不输出短语树，而输出以词为节点的有根依存图。

12.1 成分与依存的区别

成分句法关心 **span** 组成的短语层级；依存句法关心词与词之间的二元关系。一个依存弧 (h, d, l) 表示 head h 支配 dependent d ，关系标签为 l 。整个句子的依存结构通常是一棵以 ROOT 为根的有向树。

12.2 依存关系与图约束

依存图通常要求：

1. 每个非 ROOT 词恰好有一个 head。
2. ROOT 没有 head。
3. 图连通且无环。
4. 某些体系要求投射性 (projectivity)。

投射性可理解为：把词按句子顺序写在一行，依存弧画在上方时不交叉。非投射结构在自由语序语言中更常见。

12.3 Transition-based parsing

Transition-based parser 维护 **stack**、**buffer** 和 **arcs**，通过动作逐步构造树。arc-standard 系统常见动作：

动作	作用
SHIFT	buffer 首词移入 stack
LEFT-ARC	建立右侧词指向左侧词的弧，并弹出 dependent
RIGHT-ARC	建立左侧词指向右侧词的弧，并弹出 dependent

分类器根据当前配置选择下一动作。优点是速度快，常可线性时间解析；缺点是早期错误会传播。

12.4 Graph-based parsing

Graph-based parser 给所有可能弧打分，然后寻找最高分合法依存树：

$$T^* = \arg \max_T \sum_{(h,d) \in T} s(h,d).$$

非投射解析可用最大生成树算法，如 Chu-Liu/Edmonds；投射解析可用 Eisner 动态规划。

12.5 评价

依存句法常用：

- UAS (Unlabeled Attachment Score): head 是否正确。
- LAS (Labeled Attachment Score): head 和 dependency label 是否都正确。

[TIP] 本讲速查

依存句法输出词级有根树。Transition-based 是动作序列搜索，graph-based 是全局树搜索；UAS 看 head，LAS 同时看 head 和 label。

第十三章 Seq2Seq、Attention 与 Transformer

[TIP] 学习指南

本章对应 lec6_seq 与李宏毅--seq2seq_v9.pptx。核心是从等长标签序列推广到输入输出长度不同的序列生成。

13.1 Sequence-to-Sequence

Seq2Seq 任务输入 $x_{1:S}$, 输出 $y_{1:T}$, 两者长度可以不同。典型任务是机器翻译、摘要、对话、语音识别、纠错等。Encoder 把输入编码为表示, Decoder 自回归生成输出:

$$p(y_{1:T} | x_{1:S}) = \prod_{t=1}^T p(y_t | y_{<t}, x_{1:S}).$$

训练常用 teacher forcing: 第 t 步 decoder 输入 gold prefix。推理时只能输入自己已生成的 token, 因此会产生 exposure bias。

13.2 Attention for Encoder-Decoder

早期 Seq2Seq 把整个输入压进一个固定向量, 长句效果差。Attention 让 decoder 每一步动态读取 encoder hidden states:

$$\begin{aligned} e_{t,s} &= \text{score}(s_{t-1}, h_s), \\ \alpha_{t,s} &= \frac{\exp(e_{t,s})}{\sum_{s'} \exp(e_{t,s'})}, \\ c_t &= \sum_s \alpha_{t,s} h_s. \end{aligned}$$

然后用 c_t 和 decoder state 预测 y_t 。

13.3 Transformer

Transformer 抛弃 RNN recurrence, 使用 self-attention 和 feed-forward network 堆叠。Scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

除以 $\sqrt{d_k}$ 是为了避免维度变大后点积方差过大导致 softmax 饱和。

Multi-head attention:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V),$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O.$$

Encoder block: self-attention、residual、layer norm、feed-forward。Decoder block 额外包含 masked self-attention 和 cross-attention。因为 self-attention 本身不编码顺序，模型需要 positional encoding。

13.4 解码策略

Greedy decoding 每步选概率最大 token，速度快但容易错过全局更优序列。Beam search 保留 B 个高分前缀：

$$\text{score}(y_{1:t}) = \sum_{i=1}^t \log p(y_i \mid y_{<i}, x).$$

开放式生成中还会使用 sampling、top-k、top-p 等策略。课程中强调：任务目标不同，解码策略也不同；翻译等答案较确定任务常用 beam，开放生成不宜只追求最高概率。

13.5 BLEU 与生成评价

BLEU 用 n-gram precision 和 brevity penalty 评价机器翻译：

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right).$$

它便于自动评估，但不能完全衡量语义正确性、事实性和可读性。

[TIP] 本讲速查

Seq2Seq = encoder + autoregressive decoder；attention = decoder 每步读输入；Transformer = self-attention + multi-head + FFN + residual/layer norm + positional encoding。

第十四章全课程公式与算法索引

14.1 公式总表

模块	核心公式
WSD/NB	$s^* = \arg \max_s P(s) \prod_{v \in C} P(v s)$
Generative classification	$\arg \max_y p(y x) = \arg \max_y p(x y)p(y)$
Log-linear	$p(y x) = \exp(\theta^\top f(x, y)) / Z(x)$
Log-linear gradient	$\nabla = \text{empirical} - \text{expected}$
n-gram	$p(w_{1:n}) \approx \prod_i p(w_i w_{i-n+1:i-1})$
PPL	$\text{PPL} = 2^{-\frac{1}{M} \sum \log_2 p(s)}$
HMM	$p(x, y) = \pi(y_1) b_{y_1}(x_1) \prod_t a_{y_{t-1}, y_t} b_{y_t}(x_t)$
Viterbi	$\delta_t(j) = \max_i \delta_{t-1}(i) a_{ij} b_j(x_t)$
CRF	$p(y x) = \exp(\text{Score}(x, y)) / \sum_{y'} \exp(\text{Score}(x, y'))$
CKY	$\pi[i, j, A] = \max_{A \rightarrow BC, k} P(A \rightarrow BC) \pi[i, k, B] \pi[k, j, C]$
Attention	$\text{softmax}(QK^\top / \sqrt{d_k})V$

14.2 动态规划模板

Viterbi: 状态是“位置 + 当前标签”，递推取 max，回指针恢复最优路径。

Forward: 状态是“位置 + 当前标签”，递推取 sum，终止时求总 likelihood。

CKY: 状态是“span + nonterminal”，递推枚举 split point 和规则，回指针恢复最优树。

Eisner: 状态是“span + 方向 + complete/incomplete”，用于投射依存树。

14.3 易混点

易混概念	区分
生成式 vs 判别式	生成式建 $p(x, y)$; 判别式建 $p(y x)$
Viterbi vs Forward	Viterbi 求最优路径 max; Forward 求总概率 sum
MEMM vs CRF	MEMM 局部归一; CRF 全局归一
成分 vs 依存	成分看短语 span; 依存看 head-dependent
PPL vs Accuracy	PPL 评价概率模型; Accuracy 评价离散预测
Beam vs Sampling	Beam 追求高概率序列; sampling 保留多样性

14.4 最后一轮自测

1. 能否从 Bayes rule 推出 Naive Bayes 的 WSD 决策式?

2. 能否写出 log-linear 的 $Z(x)$ ，并推导梯度为什么是 empirical minus expected?
3. 能否手算 trigram 句子概率和 perplexity?
4. 能否给定 HMM 参数填 Viterbi 表，并解释与 Forward 的差别?
5. 能否把 PCFG 转成 CKY chart 的初始化、递推和终止?
6. 能否解释 label bias，并说明 CRF 如何通过全局归一缓解?
7. 能否写出 scaled dot-product attention，并说明 multi-head 和 positional encoding 的必要性?

[TIP] 复习结论

整门课的核心不是背模型名，而是建立“任务输出空间决定概率对象，概率对象决定训练目标，结构假设决定解码算法，评价指标反映任务目标”的链条。